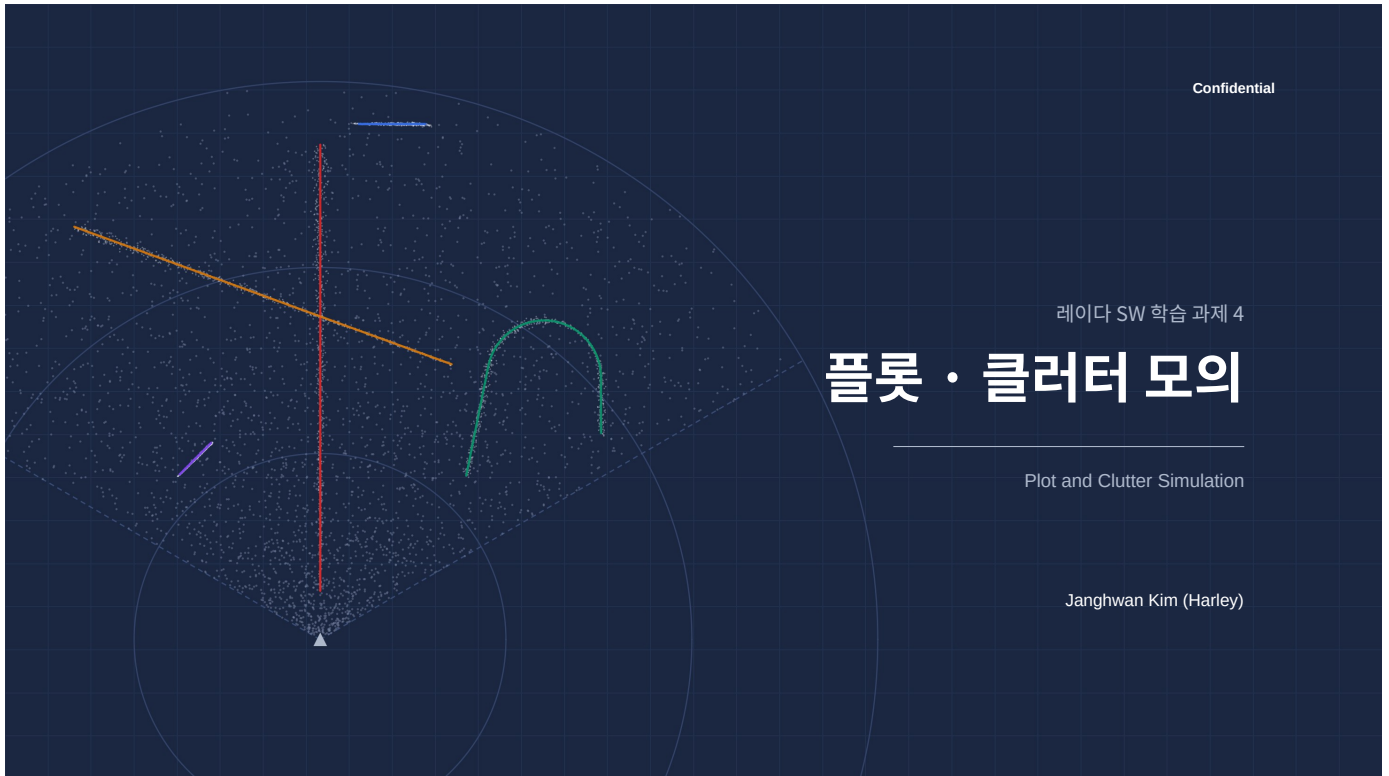




발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 플롯과 클러터 모의입니다.

지난 과제에서 만든 표적 궤적은 표적이 실제로 있던 자리, 즉 참값이었습니다. 이번에는 그 참값을 레이다가 잦 것 같은 측정값으로 바꾸고, 표적이 아닌 곳에서 생기는 가짜 탐지인 클러터도 함께 만들었습니다.

뒤에 나오는 숫자와 그림은 모두 제가 만든 프로그램의 Core 를 그대로 돌려서 얻은 값입니다.

INDEX

플롯 · 클러스터 모의
PlotSim

Janghwan Kim (Harley)

01	개요	3
02	플롯	4
03	클러스터	12
04	구현	15
05	결과	22
06	정리	27

2

발표 대본

목차 · 약 30초

순서는 여섯 부분입니다.

먼저 과제 개요를 한 장으로 보고, 플롯이 무엇이고 어떻게 모의하는지, 이어서 클러스터가 무엇이고 왜 단순하게 모의하는지 개념을 설명드리겠습니다.

그다음 C 코드로 어떻게 구현했는지 핵심 함수 위주로 보고, 실습 항목별 결과를 확인한 뒤 요구사항 대응과 가정으로 마무리하겠습니다.

01. 개요

1.1 과제 개요

과제 3 이 만든 참값 궤적을 레이다가 잰 값(플롯)으로 바꾸고, 표적이 아닌 가짜 탐지(클러터)를 함께 만듭니다.

레이다 처리 흐름에서의 위치



플롯 만들기

$$\mathbf{Z} = \mathbf{X} + \mathbf{V}$$

- 참값 $\mathbf{x}$ 에 측정 잡음 $\mathbf{v}$ 를 더한 값
- 표적마다, 스캔마다 1 개
- 거리 · 방위각 · 고각 성분별 잡음

클러터 만들기

FOV 안 무작위

- 표적과 상관없는 가짜 탐지
- 스캔마다 5 개, 매번 새로 뽑음
- 거리 · 방위각 · 고각을 균등 난수로

실습 조건

시간	60 s 동안 0.1 s 간격 → 600 스캔
레이다	과제 3 플랫폼 (32.0°N, 126.0°E, 정지)
표적	과제 3 대함 · 대공 + 3 개 = 5 개
플롯	SNR 20, 대역폭 5 MHz, K_M 1.7
빔폭	방위각 5.14°, 고각 5.31°
클러터	200 ~ 15,000 m · ±60° · 0° ~ 30°

3

발표 대본

개요 · 약 60초

과제 개요입니다. 위쪽 흐름을 보시면 레이다 데이터 처리에서 이번 과제가 어디에 있는지 알 수 있습니다.

과제 3에서는 맨 앞의 표적 궤적 모의로 표적이 실제로 있던 자리, 즉 참값을 만들었습니다. 이번 과제는 그 다음 칸인 측정 모의입니다. 여기서 만든 플롯과 클러터를 다음 과제의 추적 필터가 입력으로 받습니다.

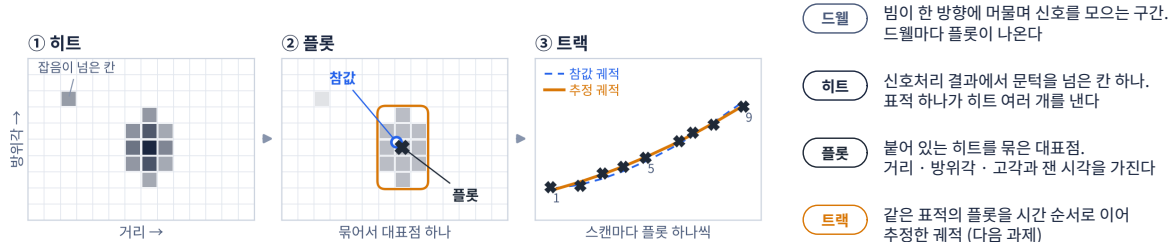
만드는 것은 두 가지입니다. 플롯은 참값 $\mathbf{x}$ 에 측정 잡음 $\mathbf{v}$ 를 더한 값이고, 표적마다 스캔마다 하나씩 나옵니다. 클러터는 표적과 상관없이 레이다 시야 안 아무 곳이나 생기는 가짜 탐지이고, 스캔마다 다섯 개씩 만들었습니다.

실습 조건은 오른쪽과 같습니다. 60 초 동안 0.1 초마다, 모두 600 스캔을 잡니다. 레이다는 과제 3의 플랫폼을 그대로 쓰고, 표적은 과제 3의 대함 · 대공 표적에 결과 화면용으로 세 개를 더해 다섯 개를 썼습니다.

02. 플롯

2.1 플롯이란

레이다가 표적을 한 번 켜 결과입니다. 빔이 한 방향에 머무는 동안(드웰) 받은 신호를 처리해 거리 · 방위각 · 고각을 얻습니다.



플롯은 측정값이라 오차가 섞여 있다. 같은 표적이라도 플롯 z 는 참값 x 와 매번 조금씩 다르다. 그 차이를 어떻게 만드는지가 이번 과제의 플롯 모의다.

4

발표 대본

플롯 · 약 60초

플롯이 무엇인지부터 보겠습니다. 플롯은 레이다가 표적을 한 번 켜 결과입니다.

빔이 한 방향에 머무는 구간을 드웰이라고 합니다. 드웰 동안 받은 신호를 처리하면 거리와 각도로 나눈 칸마다 신호 세기가 나오고, 문턱을 넘은 칸을 히트라고 합니다. 왼쪽 그림처럼 표적 하나가 히트 여러 개를 냅니다. 왼쪽 위의 칸처럼 잡음이 문턱을 넘는 경우도 있습니다.

가운데 그림처럼 붙어 있는 히트를 묶어 대표점 하나로 만든 것이 플롯입니다. 거리, 방위각, 고각과 켜 시각을 가집니다. 파란 원이 실제 표적 자리인데 플롯과 정확히 겹치지 않습니다. 측정에는 늘 오차가 있기 때문입니다.

오른쪽처럼 스캔마다 나온 플롯을 시간 순서로 이어 표적의 움직임을 추정한 것이 트랙이고, 트랙은 다음 과제에서 다룹니다.

02. 플롯

2.2 플롯 생성 과정

실제 레이다는 수신 신호를 여러 단계로 처리해 플롯을 뽑고, 모의는 그 과정을 건너뛰고 참값에 잡음을 더해 바로 플롯을 만듭니다.

(1) 실제 레이다 시스템



(2) 추적 · 성능 검증용 플롯 모의



생략해도 되는 이유 추적기가 받는 입력은 플롯뿐이다. 신호처리가 플롯에 남기는 효과를 '성분별 오차의 크기(표준편차)'로 요약해 넣으면 추적기 입장에서는 같은 성질의 입력이 된다.

5

발표 대본

플롯 · 약 75초

플롯을 만드는 과정은 두 가지입니다.

위쪽이 실제 레이다입니다. 표적에서 돌아온 신호와 수신기 잡음이 같이 들어오고, 이것을 IQ 데이터로 받습니다. 펄스 압축으로 긴 펄스를 좁게 모아 거리를 구분하고, 도플러 처리로 속도별로 나눠 움직이지 않는 클러스터를 걸러 냅니다. CFAR 로 주변 잡음 크기에 맞춘 문턱을 정하고, 문턱을 넘은 칸을 히트로 뽑습니다. 한 표적에서 나온 히트를 하나로 묶고, 모노펄스 처리로 각도를 정밀하게 구하면 드웰 플롯이 나옵니다.

아래쪽이 이번 과제에서 하는 모의입니다. 표적 정보에 잡음을 더해 바로 드웰 플롯을 만들고, 가운데 단계는 모두 생략합니다.

생략해도 되는 이유는 추적기가 받는 것이 결국 플롯뿐이기 때문입니다. 신호처리가 플롯에 남기는 효과를 거리, 방위각, 고각 오차의 크기로 요약해서 넣으면 추적기 입장에서는 같은 성질의 입력이 됩니다. 그 오차의 크기를 어떻게 정하는지 이어서 보겠습니다.

02. 플롯

2.3 플롯 모의 방법

표적 · SNR · 레이더 파라미터로 거리 · 방위각 · 고각의 표준편차를 구하고, 그 크기의 가우시안 잡음을 성분마다 더합니다.



예: 대공 표적 첫 스캔 ($t = 0$ s)의 실제 값

성분	참값 x	잡음 v	플롯 $z = x + v$
거리	13,310.2362 m	+2.4453 m	13,312.6815 m
방위각	0.0000°	-0.3574°	-0.3574°
고각	1.2315°	-0.8396°	0.3919°

고각 -0.84° 이 플롯은 고각이 작게 나와 위도 · 경도 · 고도로 바꾸면 고도 300 m 가 아니라 105 m 에 찍힌다. 이런 오차가 스캔마다 새로 생긴다.

6

발표 대본

플롯 모의 · 약 70초

플롯을 모의하는 방법은 네 단계입니다.

첫째, SNR 을 정합니다. 원래는 빔폭, 대역폭, 송신 전력 같은 레이더 파라미터와 표적 거리로 SNR 을 계산하지만, 실습은 20 으로 고정했습니다.

둘째, SNR 과 레이더 파라미터로 거리, 방위각, 고각 오차의 표준편차 p 를 구합니다. SNR 20 이면 4.74 m, 0.478 도, 0.494 도입니다.

셋째, 평균이 0 이고 표준편차가 p 인 가우시안 분포에서 잡음 v 를 성분마다 따로 뽑습니다. 넷째, 참값 x 에 잡음 v 를 더한 것이 플롯 z 입니다.

아래 표가 실제 첫 스캔의 대공 표적입니다. 참값 거리에 2.4 m 정도를 더하고, 방위각과 고각에도 각각 잡음을 더했습니다. 이 플롯은 고각이 0.84 도 작게 나와서, 위도 · 경도 · 고도로 바꾸면 고도가 300 m 가 아니라 105 m 로 찍힙니다. 이런 오차가 스캔마다 새로 생깁니다.

02. 플롯

2.4 측정 표준편차 p

세 식은 모두 '분해능 $\div \sqrt{2 \cdot \text{SNR}}$ ' 꼴입니다. 분해능은 두 표적을 따로 구분할 수 있는 최소 간격이고, 신호가 잡음보다 셀수록(SNR) 그보다 훨씬 작은 오차로 잹니다.



7

발표 대본

플롯 모의 · 약 80초

표준편차 p 를 구하는 식을 자세히 보겠습니다. 세 식은 모두 분해능을 루트 2 SNR 로 나눈 꼴입니다. 분해능은 레이다가 두 표적을 따로 구분할 수 있는 최소 간격입니다.

거리 분해능은 광속 c 를 대역폭 B 의 두 배로 나눈 값입니다. 대역폭은 송신 신호가 차지하는 주파수 폭인데, 넓을수록 펄스를 좁게 모을 수 있어서 거리를 잘게 나눌 수 있습니다. 2 로 나누는 것은 전파가 표적까지 갔다가 돌아오기 때문입니다. 대역폭 5 MHz 를 넣으면 29.98 m 입니다.

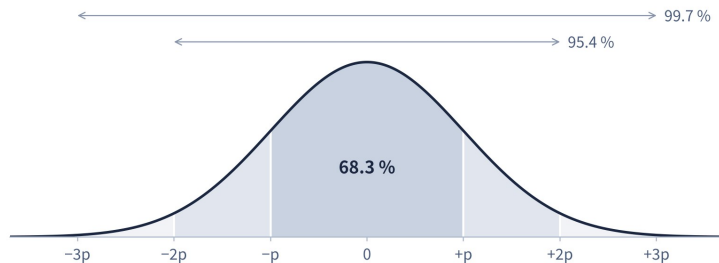
각도 분해능은 빔폭을 모노펄스 기울기 상수 K_M 으로 나눈 값입니다. 빔폭은 안테나 빔이 퍼지는 각도로, 방위각은 좌우 폭 5.14° 도, 고각은 위아래 폭 5.31° 도입니다. 모노펄스 레이다는 합 신호와 차 신호의 비로 빔 안에서 표적 각도를 다시 재는데, 그 비가 각도에 따라 변하는 기울기가 K_M 이고 1.7 을 씁니다. 그래서 각도 분해능은 3.024° 도와 3.124° 도가 됩니다.

SNR 은 표적 신호가 잡음보다 몇 배 센지를 나타내고, 실습에서는 20 입니다. 신호가 셀수록 잡음에 덜 흔들리기 때문에 분해능보다 훨씬 작은 오차로 잹 수 있습니다. 루트 2 SNR 은 루트 40, 즉 6.325 이고, 세 분해능을 모두 이 값으로 나누면 거리 4.740 m, 방위각 0.4781° 도, 고각 0.4939° 도가 나옵니다.

02. 플롯

2.5 잡음 v : 가우시안 분포

$v \sim N(0, p^2)$ 는 평균 0, 표준편차 p 인 정규분포입니다. 거리 · 방위각 · 고각 잡음을 서로 따로 뽑습니다.



성분	표준편차 p	$\pm 1p$ 안 (68.3 %)	$\pm 2p$ 안 (95.4 %)	$\pm 3p$ 안 (99.7 %)
거리	4.740 m	± 4.7 m	± 9.5 m	± 14.2 m
방위각	0.4781°	$\pm 0.48^\circ$	$\pm 0.96^\circ$	$\pm 1.43^\circ$
고각	0.4939°	$\pm 0.49^\circ$	$\pm 0.99^\circ$	$\pm 1.48^\circ$

- 표준 정규분포의 원래 모양**
 $f(t) = \exp(-t^2/2) / \sqrt{(2\pi)}$, $t = v/p$
 0 에서 가장 높고, 멀어질수록 빠르게 낮아진다
- 68 % 는 $\pm 1p$ 안**
 플롯 100 개 중 68 개는 $\pm 1p$, 95 개는 $\pm 2p$ 안.
 곡선 아래 넓이라 p 와 관계없이 늘 같은 비율
- 성분마다 따로 뽑는다**
 거리 · 방위각 · 고각 잡음은 서로 독립.
 거리 오차가 커도 각도 오차와는 무관
- 잡음은 $v = p \times t$ 로 만든다**
 표준 정규분포에서 뽑은 t 에 p 를 곱한다.
 첫 스캔 거리 잡음 $+2.445 \text{ m} = 4.740 \text{ m} \times 0.5159$

8

발표 대본

플롯 모의 · 약 70초

과제에서 정한 대로 잡음 v 는 가우시안, 즉 정규분포에서 뽑습니다. 평균이 0 이라 플롯이 어느 한쪽으로 치우치지 않고 참값 둘레에 흩어지며, 흩어지는 정도가 앞에서 구한 표준편차 p 입니다.

왼쪽 그림은 표준 정규분포 공식을 그대로 그린 것입니다. 가로축을 p 단위로 두었기 때문에 거리, 방위각, 고각이 모두 같은 모양입니다. 0 에서 가장 높고 0 에서 멀어질수록 빠르게 낮아지는 종 모양이 정규분포의 원래 모양입니다.

가장 진한 가운데 부분이 $\pm 1p$ 구간이고, 전체의 68 % 가 여기에 들어옵니다. $\pm 2p$ 안에는 95 %, $\pm 3p$ 안에는 99.7 % 가 들어옵니다. 이 비율은 곡선 아래 넓이라서 p 가 얼마든 늘 같습니다.

아래 표는 이 구간을 실제 단위로 바꾼 것입니다. 거리 오차는 68 % 가 ± 4.7 m 안에, 방위각 오차는 68 % 가 ± 0.48 도 안에 들어온다고 읽으시면 됩니다. 거의 모든 플롯은 거리로 ± 14 m, 각도로 ± 1.5 도 안에 있습니다.

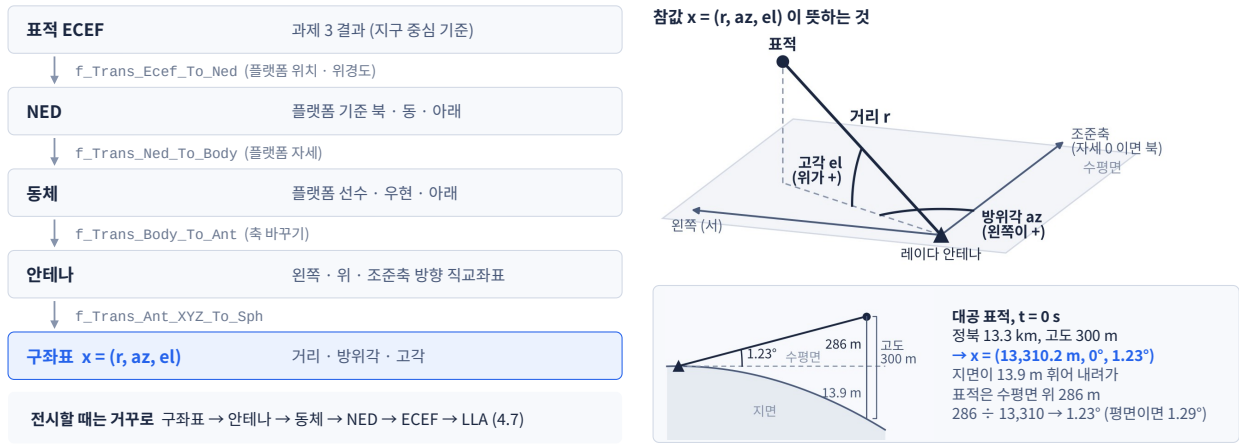
거리, 방위각, 고각 잡음은 성분마다 따로 뽑기 때문에 서로 독립입니다. 거리 오차가 크게 나왔다고 각도 오차까지 커지는 않습니다.

잡음을 실제로 만들 때는 표준 정규분포에서 수 t 를 하나 뽑아 p 를 곱합니다. 앞에서 표로 본 첫 스캔의 거리 잡음 $+2.445 \text{ m}$ 는 t 가 0.5159 로 뽑혀서, 4.740 m 에 0.5159 를 곱한 값입니다. t 를 뽑는 방법은 뒤에서 코드와 함께 보겠습니다.

02. 플롯

2.6 참값 x : 안테나 기준 거리 · 방위각 · 고각

과제 3 결과는 ECEF 위치이므로, 레이더(플랫폼) 안테나 기준의 좌표로 바꾼 뒤 잡음을 더합니다.



9

발표 대본

플롯 모의 · 약 80초

잡음을 더하려면 먼저 참값을 레이더가 재는 형태로 바꿔야 합니다. 과제 3의 결과는 지구 중심 기준의 ECEF 위치라서, 레이더 안테나 기준의 거리, 방위각, 고각으로 바꿉니다.

왼쪽 순서대로 플랫폼 위치를 기준으로 북 · 동 · 아래 좌표인 NED로 바꾸고, 플랫폼 자세로 돌려 동체 좌표로, 축을 바꿔 안테나 좌표로 옮긴 다음 거리와 두 각도로 바꿉니다. 함수는 모두 제공된 좌표변환 코드를 그대로 썼습니다.

오른쪽 그림은 이렇게 구한 세 값이 무엇인지 보여 줍니다. 거리 r 은 안테나에서 표적까지의 직선거리입니다. 방위각 az 는 수평면에서 조준축으로부터 표적 쪽으로 돌아간 각도이고, 고각 el 은 수평면에서 표적을 올려다본 각도입니다. 방위각은 조준축 왼쪽이 플러스인데, 제공된 좌표변환 코드의 규약을 그대로 따른 것입니다. 플랫폼 자세가 0이면 조준축이 북쪽이라 왼쪽은 서쪽입니다.

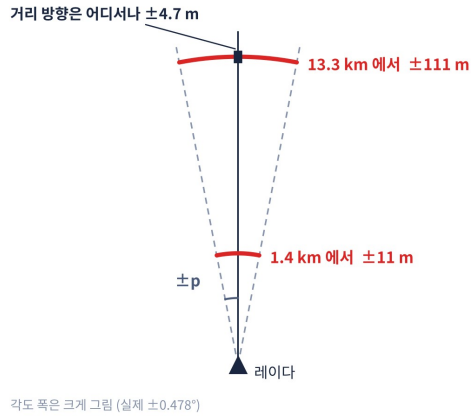
아래 상자는 대공 표적의 첫 위치입니다. 정북 13.3 km, 고도 300 m 이고, 바꾸면 거리 13,310.2 m, 방위각 0 도, 고각 1.23 도입니다. 300 을 13,310 으로 나누면 1.29 도인데 그보다 작게 나왔습니다. 상자 안 그림처럼 지구가 둥글어서, 13.3 km 앞의 지면은 레이더의 수평면보다 13.9 m 아래로 휘어 있습니다. 그래서 레이더가 보기에 표적은 수평면 위 286 m 에 있고, 286 을 13,310 으로 나누면 1.23 도가 됩니다. ECEF 에서 계산하니 이 효과가 자동으로 들어갔습니다.

전시할 때는 잡음을 더한 구좌표를 같은 길로 거꾸로 돌려 위도, 경도, 고도로 바꿉니다. ECEF 에서 바로 더하지 않고 구 좌표에서 더하는 이유는 각도 오차를 먼저 본 다음 따로 설명드리겠습니다.

02. 플롯

2.7 각도 오차는 멀수록 커집니다

방위각 · 고각 오차는 각도라서, 같은 0.478° 라도 레이더에서 멀수록 더 큰 위치 오차가 됩니다.



방향	위치 오차	13.3 km	1.4 km
거리 방향	p_{mg} 그대로	± 4.74 m	± 4.74 m
옆 (방위각)	거리 $\times p_{azi}$ [rad]	± 111 m	± 11 m
위아래 (고각)	거리 $\times p_{ele}$ [rad]	± 115 m	± 12 m

계산 예 $0.478^\circ = 0.00834$ rad $\rightarrow 13,310$ m $\times 0.00834 = 111$ m

그래서 플롯은 표적이 멀 때 옆으로 넓게 퍼지고, 다가올수록 좁아진다. 대공 표적은 13.3 km 에서 1.4 km 까지 다가오므로 이 변화가 결과 화면에 그대로 보인다 (5.5).

10

발표 대본

플롯 모의 · 약 50초

플롯이 화면에서 어떻게 보일지 미리 알려면 한 가지를 더 봐야 합니다. 방위각과 고각 오차는 각도라서, 같은 각도라도 레이더에서 멀수록 더 큰 위치 오차가 됩니다.

그림처럼 같은 $\pm p$ 라도 13.3 km 에서는 호의 길이가 ± 111 m 이고, 1.4 km 에서는 ± 11 m 입니다. 각도를 라디안으로 바꿔 거리를 곱하면 됩니다. 0.478 도는 0.00834 라디안이고, 여기에 $13,310$ m 를 곱하면 111 m 입니다.

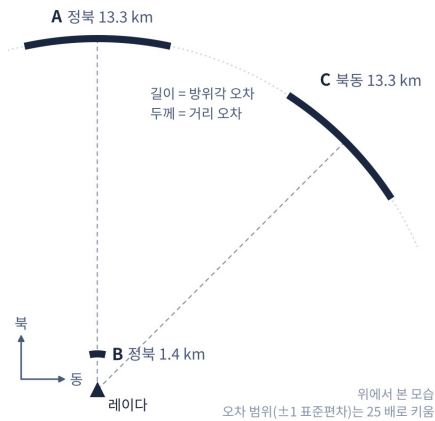
반면 거리 방향 오차는 어디서나 ± 4.74 m 로 같습니다. 그래서 먼 곳의 플롯은 거리 방향으로 거의 흔들리지 않고 옆으로 길게 퍼집니다.

대공 표적은 13.3 km 에서 출발해 1.4 km 까지 다가오므로, 플롯이 처음에는 넓게 퍼졌다가 점점 좁아질 것이라고 예상할 수 있습니다. 실제로 그런지는 뒤에서 결과로 확인하겠습니다.

02. 플롯

2.8 잡음을 구좌표에서 더하는 이유

레이더는 거리와 각도를 재므로 오차도 거리와 각도에 생깁니다. 거리 ± 4.74 m, 방위각 $\pm 0.478^\circ$, 고각 $\pm 0.494^\circ$ 이고, 이 크기는 표적이 어디에 있든 같습니다.

같은 오차를 두 좌표로 적으면 (± 1 표준편차)

표적	구좌표 (거리, 방위각)	직교좌표 (동, 북)
A 정북 13.3 km	± 4.74 m, $\pm 0.478^\circ$	동 ± 111 m, 북 ± 4.74 m
B 정북 1.4 km	± 4.74 m, $\pm 0.478^\circ$	동 ± 11 m, 북 ± 4.74 m
C 북동 13.3 km	± 4.74 m, $\pm 0.478^\circ$	동 ± 79 m, 북 ± 79 m (서로 얹힘)
	세 표적 모두 같다	표적마다 다르다

ECEF 도 직교좌표다. 축이 지구 중심 기준으로 기울어 있어 $x \cdot y \cdot z$ 가 더 섞일 뿐이다.

구좌표에서 더하면 (이번 구현) 어느 표적이든 같은 크기의 잡음 3 개를 더하면 끝난다.

ECEF 에서 더하려면 $x \cdot y \cdot z$ 에 늘 같은 크기로 더하면 A · B · C 를 다 맞출 수 없다. 표적마다 맞추려면 그 크기를 구하는 데 거리 · 방위각이 필요해서, 결국 구좌표를 거친다.

11

발표 대본

플롯 모의 · 약 80초

앞에서 참값을 구좌표로 바꾼 다음 잡음을 더한다고 했는데, 왜 ECEF 에서 바로 더하지 않는지 보겠습니다.

레이더가 직접 재는 것은 거리와 각도입니다. 그래서 오차도 거리와 각도에 생깁니다. 거리는 ± 4.74 m, 방위각은 ± 0.478 도만큼 흔들리고, 이 크기는 표적이 어디에 있든 똑같습니다.

그런데 같은 오차를 동 · 북 같은 직교좌표로 적으면 표적마다 달라집니다. 왼쪽 그림의 A 는 대공 표적이 처음 있던 정북 13.3 km 입니다. 앞에서 본 것처럼 0.478 도는 이 거리에서 옆으로 111 m 라서, 동 좌표는 ± 111 m, 북 좌표는 ± 4.74 m 흔들립니다. B 는 같은 표적이 1.4 km 까지 다가온 마지막 자리인데, 같은 각도 오차가 동 좌표로 ± 11 m 밖에 되지 않습니다. C 는 A 와 거리는 같고 북동쪽에 있는 자리입니다. 옆으로 퍼지는 111 m 가 대각선으로 놓여서 동 좌표와 북 좌표가 둘 다 ± 79 m 가 되고, 두 오차가 따로 생기지 않고 서로 얹혀서 생깁니다.

표를 보시면 구좌표 칸은 세 표적이 모두 같고, 직교좌표 칸은 표적마다 다릅니다. ECEF 도 직교좌표라서 마찬가지로이고, 축이 지구 중심 기준으로 기울어 있어서 x, y, z 가 더 섞일 뿐입니다. 고각도 같은 이야기입니다.

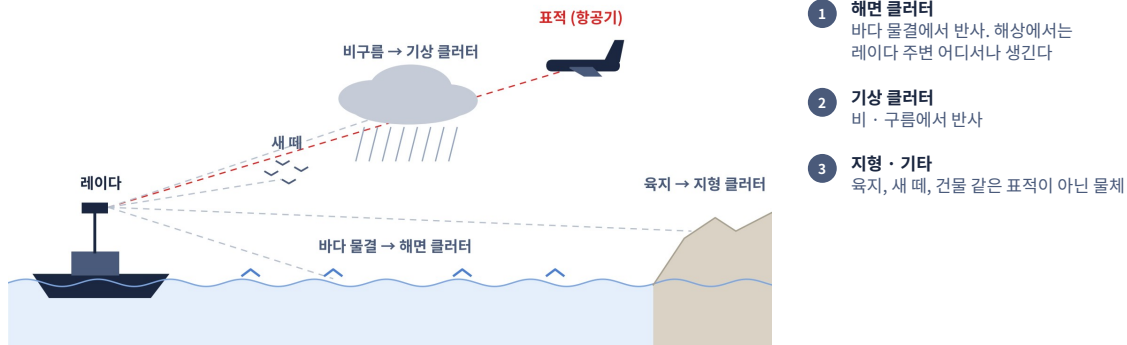
그래서 ECEF 의 x, y, z 에 늘 같은 크기의 잡음을 더하면 A, B, C 의 서로 다른 오차를 만들 수 없습니다. 표적마다 맞는 크기로 바뀌어서 더하려면 그 크기를 구하는 데 표적의 거리와 방위각이 필요하니, 결국 구좌표를 거쳐야 합니다.

구좌표에서 더하면 표적이 어디에 있든 같은 크기의 잡음 세 개를 더하면 끝납니다. 그래서 참값을 구좌표로 바꾼 다음 잡음을 더했습니다.

03. 클러터

3.1 클러터란

표적이 아닌 반사체(바다 물결, 비구름, 새, 육지)에서 돌아온 신호가 문턱을 넘어 생긴 탐지입니다. 추적기에는 가짜 플롯으로 들어옵니다.



- 1 **해면 클러터**
바다 물결에서 반사. 해상에서는 레이더 주변 어디서나 생긴다
- 2 **기상 클러터**
비 · 구름에서 반사
- 3 **지형 · 기타**
육지, 새 떼, 건물 같은 표적이 아닌 물체

추적기는 구분하지 못한다. 플롯이 표적에서 왔는지 클러터에서 왔는지 모르므로, 클러터가 섞인 입력으로 추적기를 시험해야 한다.

12

발표 대본

클러터 · 약 50초

다음은 클러터입니다. 레이더는 표적만 비추는 것이 아니라서 바다 물결, 비구름, 새 떼, 육지에서도 신호가 돌아옵니다. 이 신호가 문턱을 넘으면 표적이 아닌데도 탐지가 생기고, 이것을 클러터라고 합니다.

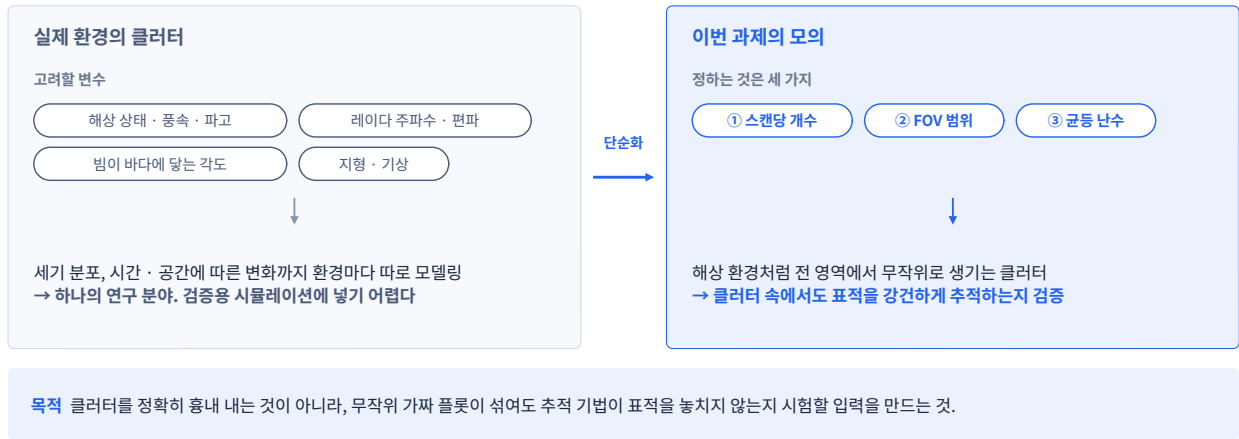
해상에서 운용하는 레이더는 바다 물결 때문에 주변 어디서나 클러터가 생길 수 있습니다.

중요한 점은 추적기 입장에서 클러터도 똑같은 플롯으로 들어온다는 것입니다. 거리, 방위각, 고각만 보고는 표적에서 온 것인지 클러터에서 온 것인지 알 수 없습니다. 그래서 클러터가 섞인 상태에서도 표적을 놓치지 않는지 추적기를 시험해야 하고, 이번 과제에서 그 입력을 만듭니다.

03. 클러스터

3.2 클러스터를 단순하게 모의하는 이유

환경에 맞는 클러스터를 그대로 모의하는 일은 그 자체로 하나의 연구 분야라서, 데이터 처리 검증용 시뮬레이션에서는 단순한 방법을 씁니다.



13

발표 대본

클러스터 · 약 50초

그렇다면 클러스터를 어떻게 모의할까요. 실제 환경의 클러스터를 그대로 만들려면 바다 상태와 바람, 레이더 주파수와 편파, 빔이 바다에 닿는 각도, 지형과 날씨까지 고려해야 합니다. 클러스터 세기가 어떤 분포를 따르는지, 시간과 공간에 따라 어떻게 변하는지도 환경마다 다릅니다. 그 자체로 하나의 연구 분야라서, 데이터 처리 기능을 검증하는 시뮬레이션에 넣기는 어렵습니다.

그래서 이번 과제는 해상 환경처럼 전 영역에서 클러스터가 무작위로 생기는 상황을 단순하게 만듭니다. 정하는 것은 스캔당 개수, 클러스터가 생길 수 있는 범위, 그리고 그 범위 안에서 고르게 뽑는 균등 난수, 이 세 가지뿐입니다.

목적은 클러스터를 정확히 흉내 내는 것이 아니라, 무작위 가짜 플롯이 섞여도 추적 기법이 표적을 놓치지 않는지 시험할 입력을 만드는 것입니다.

03. 클러스터

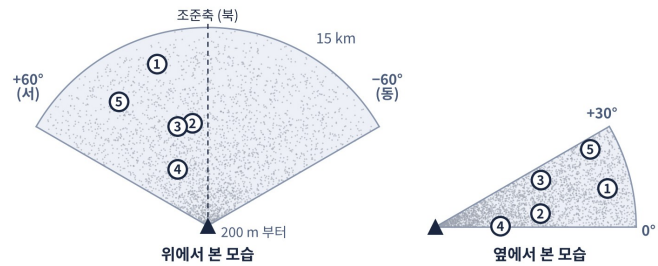
3.3 클러스터 모의 방법

스캔마다 정한 개수만큼, 거리 · 방위각 · 고각을 각각 FOV 의 min ~ max 사이에서 균등 난수로 뽑습니다.

- 1 **개수**
한 스캔에 만들 클러스터 수
실습은 5 개 (화면에서 0 ~ 20 입력)
- 2 **FOV 범위**
거리 200 ~ 15,000 m
방위각 -60° ~ +60°, 고각 0° ~ +30°
- 3 **균등 난수**
 $c = \min + (\max - \min) \times u$, $u \sim U(0, 1)$
성분마다 UNIRAN() 을 따로 부른다

첫 스캔의 1 번 클러스터

거리	$200 + 14,800 \times 0.875566 = 13,158.4 \text{ m}$
방위각	$-60 + 120 \times 0.645589 = +17.47^\circ$
고각	$0 + 30 \times 0.420927 = 12.63^\circ$



번호: 첫 스캔에서 뽑힌 클러스터 5 개 · 회색 점: 600 스캔 전체 3,000 개 클러스터는 표적과 상관없이 스캔마다 새로 뽑는다. 방위각 + 는 조준축 왼쪽(서쪽).

14

발표 대본

클러스터 · 약 60초

클러스터를 만드는 방법은 세 단계입니다.

첫째, 한 스캔에 클러스터를 몇 개 만들지 정합니다. 실습 조건은 5 개이고, 프로그램 화면에서 0 부터 20 까지 바꿀 수 있습니다.

둘째, 클러스터가 생길 수 있는 범위인 FOV 를 정합니다. 거리는 200 m 에서 15 km, 방위각은 ± 60 도, 고각은 0 도에서 30 도입니다. 오른쪽 그림의 회색 부채꼴이 이 범위입니다.

셋째, 0 과 1 사이의 균등 난수 u 를 뽑아 범위 폭에 곱하고 최솟값을 더합니다. 예를 들어 첫 스캔의 1 번 클러스터는 거리 쪽 u 가 0.875566 이 나와서 200 더하기 14,800 곱하기 u , 13,158.4 m 가 됐습니다. 방위각과 고각도 난수를 따로 뽑아 같은 방식으로 정합니다.

그림의 1 번부터 5 번이 첫 스캔에서 뽑힌 클러스터이고, 회색 점은 600 스캔 동안 뽑힌 3,000 개 전체입니다. 클러스터는 표적과 상관없이 스캔마다 새로 뽑습니다.

04. 구현

4.1 프로그램 구성

과제 3 프로그램(TargetSim)을 가져와, C Core 에 측정 모의 모듈을 넣고 MFC 화면에 입력 칸 · 난수 확인 창 · 지도 점을 더했습니다.

TargetSimCore (C, DLL)

PlotSim.c	새로 작성	참값 → 플롯, 클러스터 생성, 난수 검증
NoiseGeneration.c	제공 코드	UNIRAN, GAUSS (고치지 않음)
TargetSim.c	과제 3	표적 · 플랫폼 상태를 ECEF 로 갱신
CoordinateTransform.c	제공 코드	좌표 변환 (고치지 않음)

TargetSimUI (C++, MFC)

CPlotResult	새로 작성	600 스캔 결과와 오차 표준편차
CRandDlg · CHistView	새로 작성	난수 확인 창 (히스토그램)
CTrajectoryPlot	수정	위경도 지도에 플롯 · 클러스터 점, 범례
CSimResult	과제 3	0 ~ 60 s 표본 601 개

시나리오 실행

CSimResult::f_Run

CPlotResult::f_Run

f_Pcs_Measure

요약 줄 · 지도

화면에 더한 '측정 모의' 칸 (구성도)

측정 모의 (과제 4)

난수 확인...

SNR (선형)

20

스캔당 클러스터

5

☒ 플롯 표시

☒ 클러스터 표시

표준편차 p (이론)

거리 4.740 m, 방위각 0.4781°, 고각 0.4939°

플롯 오차 표준편차

거리 4.800 m, 방위각 0.4760°, 고각 0.4928°

(플롯 3000개, 클러스터 3000개)

■ SNR, 클러스터 수를 바꾸면 궤적은 그대로 두고 측정 모의만 다시 한다

■ '난수 확인...' → UNIRAN · GAUSS 히스토그램 창

■ 체크 상자로 지도의 플롯 · 클러스터 점을 켜고 끄다

15

발표 대본

구현 · 약 60초

이제 구현입니다. 과제 3 에서 만든 프로그램을 그대로 가져와서 필요한 부분만 더했습니다.

C 로 된 Core 에는 PlotSim.c 를 새로 넣었습니다. 참값을 구해 플롯을 만들고, 클러스터를 뽑고, 난수를 검증하는 함수가 모두 여기에 있습니다. 난수 함수와 좌표변환 함수는 제공된 코드를 고치지 않고 그대로 썼습니다.

MFC 화면 쪽에는 측정 결과를 담고 오차 통계를 내는 CPlotResult 와 난수 히스토그램을 보여 주는 난수 확인 창을 새로 만들었고, 지도에 플롯과 클러스터 점을 찍도록 그림 클래스를 고쳤습니다.

실행 순서는 아래와 같습니다. 시나리오를 실행하면 과제 3 코드가 0 초부터 60 초까지 601 개 표본을 만들고, 그중 앞의 600 개를 0.1 초 간격의 스캔으로 씁니다.

오른쪽은 화면에 더한 측정 모의 칸의 구성입니다. SNR 과 스캔당 클러스터 수를 바꾸면 표적 궤적은 그대로 두고 측정 모의만 다시 합니다. 아래 두 줄에 이론 표준편차와 실제 플롯 오차의 표준편차가 함께 나옵니다.

04. 구현

4.2 결과 구조체와 상수

TargetSimCore/include/PlotSim.h

과제 조건 값은 상수로 한곳에 모으고, 스캔 하나의 참값 · 플롯 · 클러터를 ST_PcsScan 하나에 담았습니다.

```

11 #define PCS_BANDWIDTH 5.0e6 // [Hz]
12 #define PCS_BEAM_AZI 5.14 // [deg] 방위각 빔폭
13 #define PCS_BEAM_ELE 5.31 // [deg] 고각 빔폭
14 #define PCS_KM 1.7 // 모노펄스 기술기 상수 (방위각, 고각 같음)
17 #define PCS_CLT_RNG_MIN 200.0 // [m]
18 #define PCS_CLT_RNG_MAX 15000.0
19 #define PCS_CLT_AZI_MIN (-60.0) // [deg]
20 #define PCS_CLT_AZI_MAX 60.0
21 #define PCS_CLT_ELE_MIN 0.0 // [deg]
22 #define PCS_CLT_ELE_MAX 30.0
23 #define PCS_MAX_CLUTTER 20 // 스캔당 클러터 최대 개수
24 #define PCS_SEED 12345 // NoiseGeneration.c의 SEED 초기값
...
45 typedef struct
46 {
47     FLOAT64 time; // [s]
48     INT32 nTargetNum;
49     STRUCT_Coord_Sph st_True[TGT_MAX_TARGET_NUM]; // 참값 x, [0] 이 표적 1
50     STRUCT_Coord_Sph st_Plot[TGT_MAX_TARGET_NUM]; // 플롯 z = x + v
51     STRUCT_Coord_Lla st_PlotLla[TGT_MAX_TARGET_NUM]; // 전시용. 위도, 경도 [rad], 고도 [m]
52     INT32 nClutterNum;
53     STRUCT_Coord_Sph st_Clutter[PCS_MAX_CLUTTER];
54     STRUCT_Coord_Lla st_ClutterLla[PCS_MAX_CLUTTER];
55 } ST_PcsScan;
                
```

- 1 **레이다 파라미터 (과제 조건)**
대역폭 5 MHz, 빔폭 5.14° · 5.31°, K_M 1.7
- 2 **클러터 범위 · 개수 상한 · 시드**
FOV 는 도 단위로 두고 쓸 때 라디안으로
- 3 **참값 x 와 플롯 z 를 같은 순번으로**
다음 과제에서 추적 결과를 참값과 바로 비교할 수 있다
- 4 **클러터는 표적과 따로**
스캔마다 개수만큼 구조표와 위경도

단위 거리 m, 각도 rad. 방위각은 왼쪽 +, 고각은 위 +. [0] 이 표적
 1. PCS = Plot Clutter Simulation

16

발표 대본

구현 · 약 50초

PlotSim.h 입니다. 위쪽은 상수입니다. 대역폭, 빔폭, 모노펄스 상수 같은 과제 조건과 클러터 범위, 스캔당 최대 클러터 수, 난수 시드를 한곳에 모았습니다. 앞에 붙은 PCS 는 Plot Clutter Simulation 을 줄인 것입니다.

아래는 스캔 하나의 측정 결과를 담은 ST_PcsScan 입니다. 시각과 표적 수, 표적마다 참값과 플롯, 전시용 위경도를 넣고, 클러터는 개수와 함께 따로 담습니다.

참값과 플롯을 같은 순번으로 함께 저장한 이유는 다음 과제에서 추적 결과를 참값과 바로 비교하기 위해서입니다. 각도는 라디안이고, 방위각은 왼쪽이 플러스입니다.

04. 구현

4.3 난수 함수 UNIRAN · GAUSS (제공 코드)

TargetSimCore/src/NoiseGeneration.c

제공된 코드를 고치지 않고 썼습니다. UNIRAN 은 0 ~ 1 균등난수, GAUSS 는 UNIRAN 12 개를 더해 만든 정규난수입니다.

```

7 long long int SEED = 12345;
...
19 double UNIRAN(void)
20 {
21     long long int M = 2147483647;
22     long long int a = 16807;
...
29     xn = SEED;
30     xn = abs((a * xn) % M);
31     SEED = xn;
32     ans = (double)xn / (double)M;
33     return ans;
34 }
35
36 double GAUSS(double m, double dev)
37 {
38     int i;
39     double a = 0.;
40     for (i = 0; i < 12; i++)
41     {
42         a += UNIRAN();
43     }
44     return ((a - 6.0) * dev + m);
45 }
46
47
48
49

```

1 전역 SEED 하나
이전 값으로 다음 값이 정해진다.
같은 시드 → 같은 난수열

2 $x \leftarrow 16807 \cdot x \bmod (2^{31} - 1)$
x / M 을 내보내 $0 < u < 1$
(0 과 1 은 나오지 않음)

3 UNIRAN 12 개를 더함
평균 6, 분산 $12 \times 1/12 = 1$

4 6 을 빼고 $\times dev + m$
그래서 GAUSS(0, p) 의 p 는
표준편차

UNIRAN 1개

2개 더함

12개 더하고 6 뺀

균등난수를 여러 개 더할수록 종 모양 (중심극한정리)

17

발표 대본

구현 · 약 60초

난수 함수는 제공된 코드를 그대로 썼습니다.

UNIRAN 은 이전 값에 16807 을 곱해 2 의 31 승 빼기 1 로 나눈 나머지를 다음 값으로 삼고, 그 값을 다시 2 의 31 승 빼기 1 로 나눠 0 과 1 사이의 수를 냅니다. 상태가 전역 변수 SEED 하나에 들어 있어서 시드가 같으면 늘 같은 순서의 난수가 나옵니다.

GAUSS 는 UNIRAN 을 12 번 더한 뒤 6 을 뺍니다. 균등난수 하나의 분산이 12 분의 1 이라 12 개를 더하면 분산이 정확히 1 이 되고, 평균 6 을 빼면 평균이 0 이 됩니다. 오른쪽 그림처럼 균등난수 하나는 평평하지만 두 개를 더하면 삼각형, 12 개를 더하면 거의 정규분포가 됩니다. 중심극한정리입니다.

마지막에 dev 를 곱하고 m 을 더하므로 GAUSS 의 두 번째 인자는 표준편차입니다. 그래서 플롯 잡음은 GAUSS(0, p) 로 뽑습니다.

04. 구현

4.4 표준편차 계산 (f_Pcs_CalcSigma)

TargetSimCore/src/PlotSim.c

SNR 이 고정이라 측정을 시작할 때 한 번만 계산합니다. 빔폭은 도 단위라 라디안으로 바꿔 넣습니다.

```

90 // 측정 표준편차 p.snr 은 선형값.
91 // p_rng = c / (2 * B * sqrt(2 * SNR)), p_azi = 빔폭 / (K_M * sqrt(2 * SNR)), p_ele 도 같은 식.
92 STRUCT_Coord_Sph f_Pcs_CalcSigma(FLOAT64 snr)
93 {
94     const FLOAT64 root = sqrt(2.0 * snr);
95     STRUCT_Coord_Sph st_Sigma;
96
97     st_Sigma.r = LIGHT_SPEED / (2.0 * PCS_BANDWIDTH * root);
98     st_Sigma.az = f_Deg_To_Rad(PCS_BEAM_AZI) / (PCS_KM * root);
99     st_Sigma.el = f_Deg_To_Rad(PCS_BEAM_ELE) / (PCS_KM * root);
100
101     return st_Sigma;
102 }

```

- 1 $\sqrt{2 \cdot \text{SNR}}$ 를 한 번
SNR 20 $\rightarrow \sqrt{40} = 6.325$
- 2 거리: $c / (2B \cdot \sqrt{2 \cdot \text{SNR}})$
= 4.740 m
- 3 각도: 빔폭[rad] / $(K_M \cdot \sqrt{2 \cdot \text{SNR}})$
= 8.344 mrad (0.4781°)
= 8.620 mrad (0.4939°)
- 4 SNR 이 고정이라 처음 한 번
스캔마다 다시 구하지 않는다

호출: f_Pcs_Measure 시작에서 한 번

```

106 VOID f_Pcs_Measure(ST_PcsScan *st_Scan, const ST_SimSample *st_Sample,
107                   INT32 nScanNum, FLOAT64 snr, INT32 nClutterNum)
108 {
109     const STRUCT_Coord_Sph st_Sigma = f_Pcs_CalcSigma(snr);

```

단위 주의 참값 각도가 라디안이므로 p 도 라디안으로 맞춘다. 도 단위 0.478 을 그대로 더하면 오차가 57 배가 된다.

18

발표 대본

구현 · 약 40초

표준편차를 계산하는 함수입니다. SNR 이 고정이므로 측정을 시작할 때 한 번만 부릅니다.

루트 2 SNR 을 먼저 구해 두고, 거리는 광속을 대역폭의 두 배와 이 값의 곱으로 나눕니다. 각도는 빔폭을 라디안으로 바꾼 뒤 K_M 과 이 값의 곱으로 나눕니다.

각도를 라디안으로 바꾸는 이유는 참값 방위각과 고각이 라디안이기 때문입니다. 도 단위 0.478 을 라디안 각도에 그대로 더하면 오차가 57 배로 커집니다. 그래서 표준편차는 이 함수 안에서 라디안으로 바꿔 내보냅니다.

결과는 거리 4.740 m, 방위각 0.4781 도, 고각 0.4939 도로, 앞에서 직접 계산한 값과 같습니다.

04. 구현

4.5 플롯 생성 (f_Pcs_Measure 앞부분)

TargetSimCore/src/PlotSim.c

스캔마다 표적의 참값을 안테나 좌좌표로 바꾸고, 거리 · 방위각 · 고각에 가우시안 잡음을 하나씩 더해 플롯을 만듭니다.

117	nClutterNum = (nClutterNum < 0) ? 0 :	1	클러터 수 0 ~ 20, 시드 12345
118	((nClutterNum > PCS_MAX_CLUTTER) ? PCS_MAX_CLUTTER : nClutterNum);		실행할 때마다 같은 결과
118	changeSEED(PCS_SEED);		
120	for (nScan = 0; nScan < nScanNum; nScan++)	2	스캔 k = 과제 3 표본 k
121	{		t = k × 0.1 s
122	st_In = &st_Sample[nScan];		레이다는 그 시각의 플랫폼
123	st_Radar = &st_In->st_Platform;		
124	st_Out = &st_Scan[nScan];		
126	st_Out->time = st_In->simTime;	3	참값 x
127	st_Out->nTargetNum = st_In->nTargetNum;		ECEF → NED → 동체 → 안테나
129	for (nTarget = 0; nTarget < st_In->nTargetNum; nTarget++)		→ 좌좌표 (2.6)
130	{		
131	// 참값 x	4	플롯 z = x + GAUSS(0, p)
132	st_Out->st_True[nTarget] =		한 줄에 난수 하나
	f_Pcs_EcefToSph(&st_In->st_Target[nTarget].st_PosEcef, st_Radar);		거리 → 방위각 → 고각 순서가 고정
134	// 플롯 z = x + v, v ~ N(0, p^2)	5	전사용 LLA
135	st_Out->st_Plot[nTarget].r = st_Out->st_True[nTarget].r + GAUSS(0.0, st_Sigma.r);		플롯을 위도 · 경도 · 고도로
136	st_Out->st_Plot[nTarget].az = st_Out->st_True[nTarget].az + GAUSS(0.0, st_Sigma.az);		되돌려 저장 (4.7)
137	st_Out->st_Plot[nTarget].el = st_Out->st_True[nTarget].el + GAUSS(0.0, st_Sigma.el);		
138	st_Out->st_PlotLLa[nTarget] = f_Pcs_SphToLla(&st_Out->st_Plot[nTarget], st_Radar);		
139	}		

19

발표 대본

구현 · 약 60초

측정 함수 f_Pcs_Measure 의 앞부분입니다.

표준편차는 앞에서 본 대로 처음에 한 번 구해 두고, 클러터 수를 0 에서 20 사이로 맞춘 다음 시드를 12345 로 되돌립니다. 그래서 측정을 몇 번 다시 해도 결과는 늘 같게 나옵니다.

스캔 루프는 과제 3 이 만든 표본을 하나씩 받습니다. k 번째 스캔은 k 곱하기 0.1 초 시각의 표본이고, 레이더 위치와 자세는 그 시각의 플랫폼 값을 씁니다.

표적마다 먼저 참값을 안테나 기준 좌좌표로 바꾸고, 거리, 방위각, 고각에 GAUSS 로 뽑은 잡음을 하나씩 더합니다. 한 줄에 난수를 하나씩만 부르게 해서 난수를 쓰는 순서가 항상 거리, 방위각, 고각으로 고정됩니다.

마지막으로 플롯을 위도, 경도, 고도로 되돌려 지도에 찍을 값으로 저장합니다. 이 변환은 뒤에서 따로 보겠습니다.

04. 구현

4.6 클러스터 생성 (f_Pcs_Measure 뒷부분)

TargetSimCore/src/PlotSim.c

표적 플롯을 모두 만든 뒤 같은 스캔 안에서 클러스터를 뽑습니다. 거리 · 방위각 · 고각마다 균등 난수를 하나씩 씁니다.

```

141 // 클러터: FOV 안 균등분포, 매 스캔 새로 뽑음
142 st_Out->nClutterNum = nClutterNum;
143 for (nClutter = 0; nClutter < nClutterNum; nClutter++)
144 {
145     st_Clutter = &st_Out->st_Clutter[nClutter];
146     st_Clutter->r = PCS_CLT_RNG_MIN +
147         ((PCS_CLT_RNG_MAX - PCS_CLT_RNG_MIN) * UNIRAN());
148     st_Clutter->az = f_Deg_To_Rad(PCS_CLT_AZI_MIN +
149         ((PCS_CLT_AZI_MAX - PCS_CLT_AZI_MIN) * UNIRAN()));
150     st_Clutter->el = f_Deg_To_Rad(PCS_CLT_ELE_MIN +
151         ((PCS_CLT_ELE_MAX - PCS_CLT_ELE_MIN) * UNIRAN()));
152     st_Out->st_ClutterLla[nClutter] = f_Pcs_SphToLla(st_Clutter, st_Radar);
153 }
154 }

```

- 클러스터 수는 화면 입력값
매 스캔 같은 수 (실습 5 개)
- 성분마다 UNIRAN() 따로
 $c = \min + (\max - \min) \times u$ (3.3)
각도는 도로 뽑아 라디안으로
- 위도 · 경도 · 고도도 저장
플롯과 같은 변환 함수 (4.7)

한 스캔에서 난수를 쓰는 순서

칸 하나 = 성분 하나 (거리 · 방위각 · 고각)



순서가 고정 시드와 순서가 같으니 늘 같은 결과가 나온다. 클러스터를 바꾸면 한 스캔에서 쓰는 난수 개수가 달라져 두 번째 스캔부터 플롯 잡음도 바뀐다.

발표 대본

구현 · 약 50초

f_Pcs_Measure 의 뒷부분, 클러스터를 만드는 코드입니다. 표적 플롯을 모두 만든 다음, 같은 스캔 안에서 클러스터를 뽑습니다.

클러스터 수는 화면에 입력한 값을 그대로 쓰고, 클러스터마다 거리, 방위각, 고각을 UNIRAN 으로 하나씩 따로 뽑습니다. 식은 앞에서 본 최솟값 더하기 범위 폭 곱하기 u 그대로이고, 각도는 도 단위로 뽑은 다음 라디안으로 바꿔 저장합니다. 마지막 줄에서 플롯과 같은 함수로 위도, 경도, 고도도 구해 줍니다.

아래 그림은 한 스캔에서 난수를 쓰는 순서입니다. 표적 다섯 개의 플롯에 GAUSS 를 15 번 쓰고, 이어서 클러스터 다섯 개에 UNIRAN 을 15 번 씁니다. GAUSS 한 번이 UNIRAN 12 번이므로 한 스캔에 UNIRAN 을 195 번 쓰는 셈입니다. 시드와 순서가 고정이라 몇 번을 실행해도 결과가 같습니다.

반대로 클러스터 수를 바꾸면 한 스캔에서 쓰는 난수 개수가 달라져서, 두 번째 스캔부터는 플롯 잡음도 함께 바뀝니다. 플롯과 클러스터가 같은 난수열을 나눠 쓰기 때문입니다.

04. 구현

4.7 전시용 LLA 변환 (f_Pcs_SphToLla)

TargetSimCore/src/PlotSim.c

플롯 · 클러터는 레이더 기준 좌표표라 지도에 바로 찍을 수 없습니다. 참값을 구할 때의 변환을 거꾸로 알아 위도 · 경도 · 고도로 바꿉니다.



21

발표 대본

구현 · 약 50초

플롯과 클러터는 레이더 기준의 거리와 각도라서 그대로는 지도에 찍을 수 없습니다. 그래서 f_Pcs_SphToLla 로 위도, 경도, 고도로 바꿉니다.

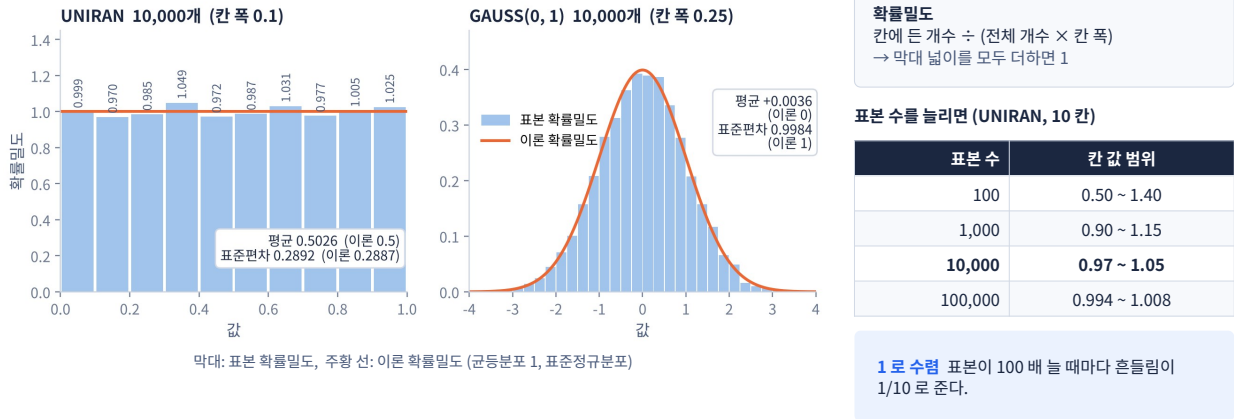
순서는 참값을 구할 때의 반대입니다. 좌표표를 안테나 직교좌표로 바꾸고, 축을 바꿔 동체 좌표로, 플랫폼 자세로 NED로, 플랫폼 위치로 ECEF로 옮긴 다음 마지막에 위도, 경도, 고도를 얻습니다. 코드의 번호와 오른쪽 그림의 번호가 같은 단계입니다. 자세와 위치는 그 스캔 시각의 플랫폼 값을 쓰고, 변환 함수는 모두 제공된 좌표변환 코드입니다.

예를 들어 첫 스캔 대공 표적의 플롯은 거리 13,312.7 m, 방위각 -0.357 도, 고각 0.392 도인데, 이것을 바꾸면 북위 32.12005 도, 동경 126.00088 도, 고도 105 m 가 됩니다. 방위각이 마이너스라 참값보다 조금 동쪽에 찍혔고, 고각이 작게 나와 참값 300 m 보다 낮게 찍혔습니다. 앞에서 표로 본 그 플롯입니다.

05. 결과

5.1 난수 확인 (실습 2.1)

프로그램의 '난수 확인' 창과 같은 계산입니다. 시드 12345 에서 UNIRAN 과 GAUSS(0, 1) 를 각각 10,000 개 뽑아 확률밀도로 나타냈습니다.



22

발표 대본

결과 · 약 60초

결과입니다. 먼저 실습 2.1의 난수 확인입니다. 프로그램의 난수 확인 창과 같은 계산으로, 시드 12345에서 각각 10,000개를 뽑았습니다.

막대 높이는 확률밀도입니다. 칸에 든 개수를 전체 개수와 칸 폭으로 나눈 값이라 막대 넓이를 모두 더하면 1이 됩니다.

왼쪽 UNIRAN은 열 칸 모두 0.97에서 1.05 사이로 이론값 1 근처에 있습니다. 평균 0.5026, 표준편차 0.2892로 이론값 0.5, 0.2887과 거의 같습니다.

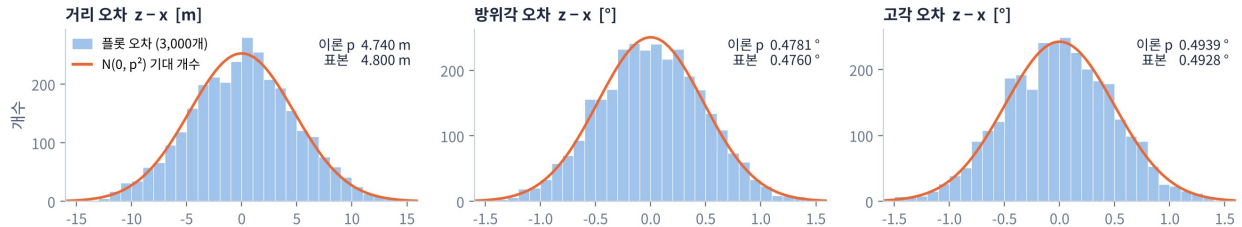
과제에서 1에 수렴하는지 확인하라고 했으므로 표본 수를 바꿔 봤습니다. 100개일 때는 0.5에서 1.4까지 흔들리지만 10만개면 0.994에서 1.008로 좁아집니다. 표본이 100배 늘 때 흔들림이 10분의 1로 줄면서 1에 수렴합니다.

오른쪽 GAUSS는 막대가 표준정규 곡선과 겹치고, 평균 0.0036, 표준편차 0.9984로 평균 0, 표준편차 1을 잘 따릅니다.

05. 결과

5.2 플롯 오차 확인 (실습 2.2)

표적 5 개 × 600 스캔 = 플롯 3,000 개의 오차 $z-x$ 를 성분별로 모았습니다. 분포 모양과 표준편차가 모두 이론값과 맞습니다.



성분	이론 μ	플롯 오차 표준편차	비율 (표본 / 이론)
거리	4.740 m	4.800 m	1.013
방위각	0.4781°	0.4760°	0.996
고각	0.4939°	0.4928°	0.998

- 3,000 개로 구한 표준편차는 그 자체로 약 1.3 % 흔들린다 → 세 값 모두 그 범위 안
- 세 성분 사이 상관계수 $-0.03 \sim +0.01$ → 성분끼리 상관이 없다
- 화면 요약 줄의 '플롯 오차 표준편차' 와 같은 값

23

발표 대본

결과 · 약 60초

다음은 실습 2.2, 플롯입니다. 표적 다섯 개를 600 스캔 동안 재서 나온 플롯 3,000 개의 오차, 즉 플롯에서 참값을 뺀 값을 성분별로 모았습니다.

막대는 실제 오차의 개수이고, 주황 선은 표준편차 σ 인 정규분포라면 기대되는 개수입니다. 세 성분 모두 막대가 곡선을 잘 따라갑니다.

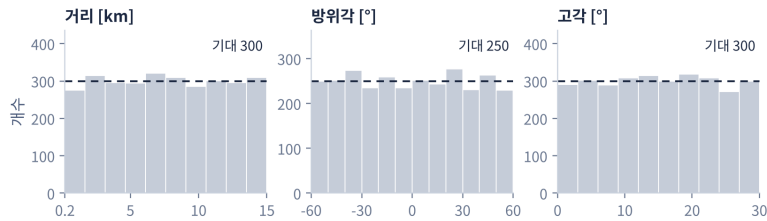
표준편차를 보면 거리 4.800 m, 방위각 0.4760 도, 고각 0.4928 도로, 이론값과의 비가 1.013, 0.996, 0.998 입니다. 3,000 개로 구한 표준편차는 그 자체로 1.3 % 정도 흔들리기 때문에 이 정도 차이는 정상입니다.

세 성분 오차끼리의 상관계수도 0 에 가까워서, 성분마다 따로 뽑은 잡음이 서로 섞이지 않았다는 것도 확인했습니다.

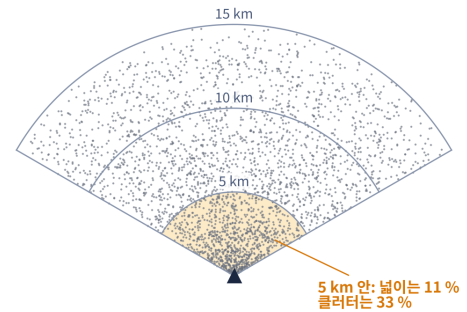
05. 결과

5.3 클러스터 분포 확인 (실습 2.3)

클러스터 3,000 개 (600 스캔 × 5 개) 가 모두 FOV 안에 있고, 거리 · 방위각 · 고각이 각 범위에서 고르게 나왔습니다.



성분	FOV	뽑힌 범위	평균 (기대)
거리	200 ~ 15,000 m	206 ~ 14,995 m	7,645 m (7,600)
방위각	-60° ~ +60°	-60.0° ~ +60.0°	-0.39° (0)
고각	0° ~ +30°	0.0° ~ 30.0°	15.01° (15)



거리를 고르게 뽑으면 거리 칸마다 개수는 같지만 가까운 칸일수록 넓이가 좁다. 그래서 레이더 가까이 촘촘하다 (오류 아님).

24

발표 대본

결과 · 약 50초

실습 2.3 클러스터입니다. 600 스캔 동안 5 개씩, 3,000 개의 클러스터를 거리, 방위각, 고각별로 나눠 셧습니다.

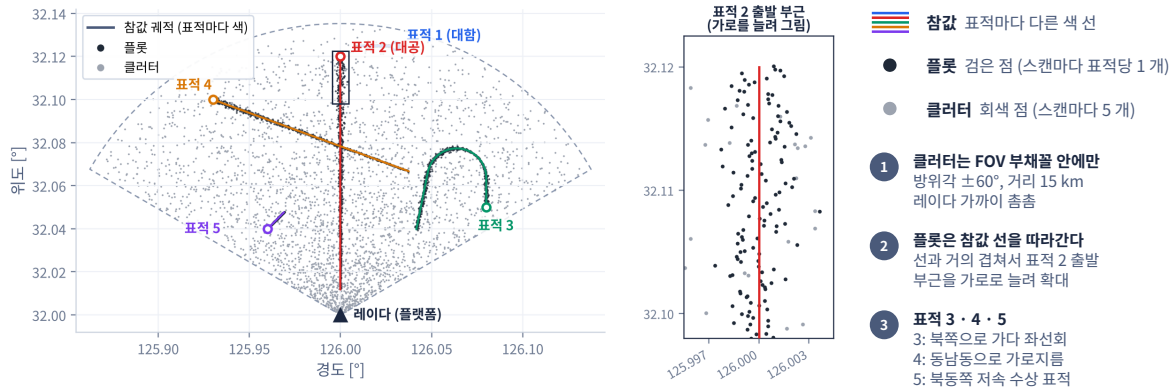
점선이 균등분포일 때 칸마다 기대되는 개수인데, 세 성분 모두 막대가 그 근처에 고르게 있습니다. 가장 작은 값과 큰 값도 정해 둔 범위 안이라 FOV 밖으로 나간 클러스터는 하나도 없습니다.

오른쪽 그림을 보시면 클러스터가 레이더 가까이 촘촘합니다. 거리를 고르게 뽑았기 때문에 거리 칸마다 개수는 같은데, 가까운 칸일수록 넓이가 좁기 때문입니다. 5 km 안은 부채꼴 넓이로는 11 % 지만 클러스터는 33 % 가 들어 있습니다. 구좌표에서 균등하게 뽑은 결과이고 오류는 아닙니다.

05. 결과

5.4 표적 5개 모의 결과 (위도 · 경도 전시)

60 s 동안 0.1 s 마다 만든 600 스캔을 위도 · 경도 위에 모두 겹쳤습니다. 선은 참값, 검은 점은 플롯, 회색 점은 클러터입니다.



그림은 프로그램과 같은 Core 출력 (시드 12345, 600 스캔) 을 위도 · 경도 축에 그린 것

25

발표 대본

결과 · 약 70초

이 그림이 과제에서 요구한 결과 화면입니다. 표적 다섯 개를 60 초 동안 0.1 초마다 켜 600 스캔을 위도와 경도 위에 모두 겹쳤습니다.

색으로 세 가지를 구분했습니다. 표적마다 다른 색의 선이 참값 궤적이고, 검은 점이 플롯, 회색 점이 클러터입니다.

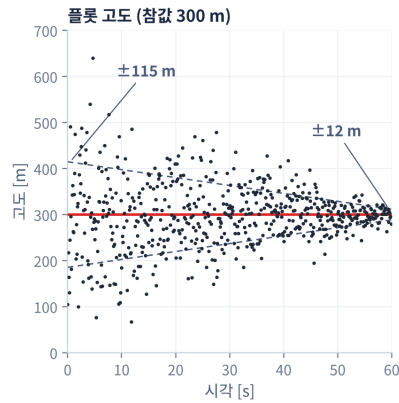
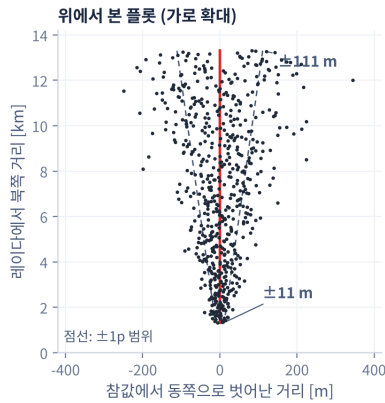
표적 1 과 2 는 과제 3 의 대함, 대공 표적이고, 3 번은 북쪽으로 가다가 좌선회, 4 번은 레이더 앞을 동남동으로 가로지르고, 5 번은 북동쪽으로 천천히 움직이는 수상 표적입니다.

회색 클러터는 방위각 $\pm 60^\circ$ 도, 거리 15 km 부채꼴 안에만 찍혔고 레이더 가까이 촘촘합니다. 검은 플롯은 참값 선을 따라 붙어 다니는데, 이 축척에서는 선과 거의 겹쳐 보여서 오른쪽에 표적 2 출발 부근을 가로로 늘려 확대했습니다. 참값 선 양쪽으로 플롯이 흩어진 모습이 보입니다.

05. 결과

5.5 대공 표적의 플롯

과제가 지목한 대공 표적(표적 2)만 보면, 각도 오차 때문에 플롯이 멀리서는 넓게, 가까이서는 좁게 흩어집니다.



- 1 **옆 방향**
13.3 km: $\pm 111\text{ m} \rightarrow 1.4\text{ km}$: $\pm 11\text{ m}$
- 2 **고도**
참값 300 m 둘레로 $\pm 115\text{ m} \rightarrow \pm 12\text{ m}$
- 3 **점선($\pm 1p$) 안에 든 비율**
600 개 중 옆 65.5 %, 고도 69.0 %
(정규분포 68.3 %)
- 4 **거리 방향 오차**
 $\pm 4.74\text{ m}$ 라 이 축척에서는 거의 보이지 않는다

26

발표 대본

결과 · 약 60초

과제가 지목한 대공 표적만 따로 보겠습니다. 왼쪽은 위에서 본 플롯인데, 옆 방향 오차를 보기 위해 가로를 크게 늘렸습니다.

앞에서 예상한 대로 처음 13 km 거리에서는 플롯이 옆으로 $\pm 111\text{ m}$ 정도 퍼지고, 1.4 km 까지 다가오면 $\pm 11\text{ m}$ 로 좁아집니다. 점선이 거리 급하기 각도 표준편차로 그린 $\pm 1p$ 범위입니다.

오른쪽은 플롯의 고도입니다. 참값은 300 m 로 일정한데, 고각 오차 때문에 처음에는 $\pm 115\text{ m}$, 끝에서는 $\pm 12\text{ m}$ 정도로 흩어집니다.

600 개 중 점선 안에 든 비율은 옆 방향 65.5 %, 고도 69.0 % 로 정규분포의 68.3 % 와 맞습니다. 거리 방향 오차 4.74 m 는 이 축척에서는 거의 보이지 않습니다.

06. 정리

6.1 과제 요구사항과 구현

실습 항목마다 어디에 구현했고 무엇으로 확인했는지 정리했습니다.

과제 요구	구현	확인 결과
UNIRAN 10,000 개, 확률밀도가 1로 수렴	f_Pcs_TestRand, 난수 확인 창	칸 값 0.970 ~ 1.049, N 늘리면 1로
GAUSS(0, 1) 10,000 개, 표준정규분포	같은 창, 표준정규 곡선 겹침	평균 +0.0036, 표준편차 0.9984
과제 3 대공 표적 · 플랫폼 위치 이용	Part 3 Core 표본 → f_Pcs_EcefToSph	t = 0: 13,310.2 m, 0°, 1.23°
SNR 20 고정, 표준편차 추출	f_Pcs_CalcSigma (화면 SNR 20)	4.740 m, 0.4781°, 0.4939°
가우시안 랜덤변수로 플롯 모의	z = x + GAUSS(0, p), 성분마다	오차 표준편차 4.800 m, 0.4760°, 0.4928°
균등 랜덤변수로 스캔당 클러스터 5 개	화면 '스캔당 클러스터' 5	600 스캔 × 5 = 3,000 개
클러스터 범위 200 ~ 15,000 m, ±60°, 0 ~ 30°	PCS_CLT_* 상수	3,000 개 모두 범위 안
60 s, 0.1 s 간격	표본 601 개 중 앞 600 개를 스캔으로	t = 0.0 ~ 59.9 s
LLA 전시, 심볼 색 구분	f_Pcs_SphToLla, CTrajectoryPlot	참값 색 선 · 플롯 검정 · 클러스터 회색
5 개 표적 결과 화면	'기분 시나리오 + 표적 3개' 프리셋	프로그램 첫 화면

27

발표 대본

정리 · 약 50초

과제 요구사항과 구현을 한 장으로 정리했습니다.

난수는 UNIRAN 과 GAUSS 를 각각 10,000 개 뽑아 분포를 확인했고, 플롯은 과제 3 의 대공 표적과 플랫폼 위치로 참값을 구해 SNR 20 표준편차의 가우시안 잡음을 더했습니다. 클러스터는 정해진 범위 안에서 스캔당 5 개를 균등 난수로 뽑았고, 60 초 동안 0.1 초 간격으로 600 스캔을 모의했습니다.

결과는 위도, 경도 위에 참값, 플롯, 클러스터를 색으로 구분해 전시했고, 오른쪽 열의 숫자로 각 항목을 확인했습니다.

06. 정리

6.2 가정과 다음 과제

모의에서 정한 것을 밝히고, 이번 결과가 다음 과제인 추적으로 어떻게 이어지는지 정리했습니다.

이번 모의에서 정한 것

- 1 SNR 은 20 으로 고정
모든 표적 · 스캔에 같은 값 → 표준편차 p 도 늘 같다
- 2 방위각은 조준축 왼쪽(서쪽)이 +
제공된 좌표변환 코드의 규약
- 3 플롯은 매 스캔 표적마다 1 개
탐지확률 1, 표적에는 FOV 판정을 하지 않음
- 4 클러터는 스캔마다 정확히 5 개
표적과 무관하게 매번 새로 뽑음
- 5 고도 0 m 표적(1, 5)의 음수 고도 플롯은 그대로
참 고각이 0° 보다 조금 낮아 절반쯤이 음수. 자르면 오차 분포가 바뀐다

다음 과제 (추적) 로 넘기는 것

- R 측정 잡음 공분산
 $R = \text{diag}(p_{\text{rng}}^2, p_{\text{azi}}^2, p_{\text{ele}}^2)$
- λ 클러터 밀도
스캔당 5 개 ÷ FOV 크기
→ PDA 같은 자료 연관 필터가 쓰는 λ
- x 침값을 함께 저장
ST_PcsScan 에 침값과 플롯이 같은 순번
→ 추적 오차를 바로 계산

28

발표 대본

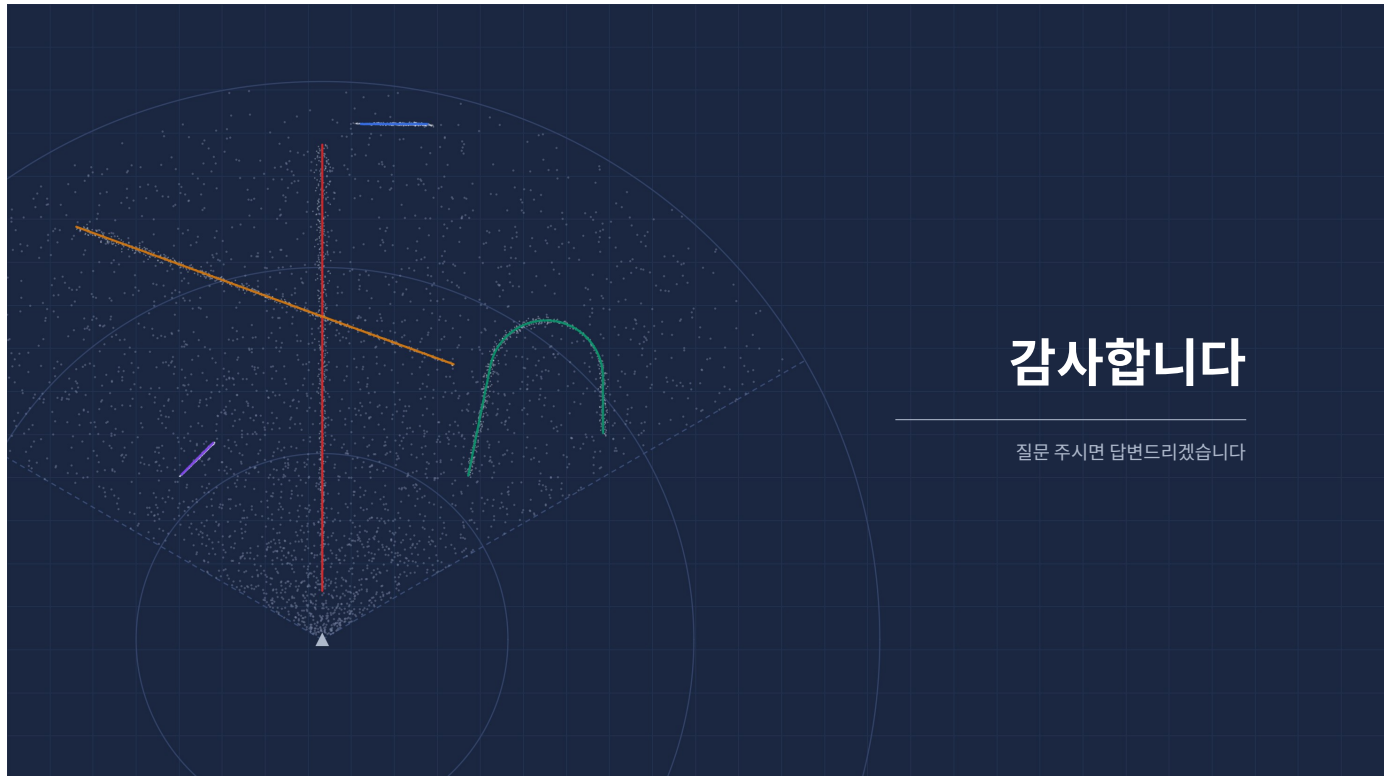
정리 · 약 50초

마지막으로 이번 모의에서 정한 가정입니다. SNR 은 모든 표적과 스캔에서 20 으로 고정했기 때문에 표준편차도 늘 같은 값입니다. 방위각은 제공된 좌표변환 코드의 규약대로 왼쪽이 플러스입니다.

플롯은 매 스캔 표적마다 하나씩 반드시 나온다고 두었고, 클러터는 정확히 5 개씩 만들었습니다. 고도 0 m 인 표적 1 과 5 는 지구 곡률 때문에 참 고각이 0 도보다 조금 낮아서 플롯의 절반쯤이 고도 음수로 나오는데, 오차 분포를 지키려고 자르지 않았습니다.

다음 과제인 추적과의 연결도 정리했습니다. 이번에 구한 표준편차 세 개는 추적 필터의 측정 잡음 공분산 R 이 되고, 스캔당 클러터 수를 FOV 크기로 나눈 값은 PDA 같은 자료 연관 필터가 쓰는 클러터 밀도 λ 가 됩니다. 결과 구조체에 침값을 함께 저장해 두었으므로 추적 오차도 바로 계산할 수 있습니다.

이상으로 발표를 마치겠습니다.

**발표 대본**

마무리 · 약 10초

감사합니다. 질문 주시면 답변드리겠습니다.